BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

- **Underlying Structure:**
  - **State Space:** Is a **graph** that can contain cycles, loops, and multiple pathways between the same nodes.
  - **Search Tree:** Is a **hierarchical tree structure** with a root and branches that strictly **cannot contain cycles**.
- **Existence and Origin:**
  - **State Space:** Exists **independently** as the complete layout/rules of the environment (e.g., all valid grid coordinates and walls).
  - **Search Tree:** Is **dynamically generated** by the search algorithm (like BFS or DFS) during execution as it explores paths.
- **State Uniqueness:**
  - **State Space:** Each physical state (like a specific (x, y) grid coordinate) exists only **once**.
  - **Search Tree:** The exact same physical state can appear **multiple times** across different branches or depths, representing different sequences of actions used to reach that location.
- **Purpose:**
  - **State Space:** Represents *where* the agent can move in the world.
  - **Search Tree:** Represents the algorithm's *memory and exploration process* of trying to find a path to the goal.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

Purpose of the reached Set

- **Prevents Infinite Loops and Cycles:** Grid environments contain numerous cycles (e.g., moving Up from (0,0) to (0,1) and then Down returns the agent back to (0,0)). The reached set tracks all states that have already been generated or expanded, ensuring the algorithm never re-enters a loop of revisiting the exact same coordinates.
- **Converts Tree Search to Graph Search:** It stops the algorithm from redundantly exploring the same state multiple times via different paths, saving critical computational memory and processor time.

What Happens to a DFS Agent Without It?

- **Endless Cycling:** Because Depth-First Search (DFS) aggressively explores the deepest available branch, without a reached set, it will get permanently trapped cycling back and forth between adjacent nodes (e.g., moving endlessly between cell A and cell B).
- **Failure to Find Goals or Terminate:** On a grid with cycles, the algorithm will fail to systematically cover the state space. It will either run indefinitely until it crashes due to a memory overflow or fail to find the goal because it never breaks out of its local cyclic loop.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS vs. DFS: Path Comparison and Optimality Analysis

1. Visual Comparison of Paths

- **BFS (Breadth-First Search):** Produces smooth, direct, and shortest-possible paths. The agent moves purposefully toward the target food pellet with minimal deviation.
- **DFS (Depth-First Search):** Produces erratic, winding, and convoluted paths. The agent often zigzags across distant parts of the grid, taking massive detours before eventually stumbling upon the goal.

2. Why BFS is Guaranteed to be Optimal

- **Level-by-Level Exploration:** BFS expands nodes in order of their distance (depth) from the starting position. It explores all paths of length 1, then length 2, then length 3, and so on.
- **Uniform Step Costs:** In this grid environment, every movement (Up, Down, Left, Right) has an identical cost of 1.
- **Guarantee:** Because BFS checks shorter paths before longer paths, the **very first time** it encounters the goal state, that path is mathematically guaranteed to have the minimum number of steps.

3. Why DFS Produces Suboptimal Routes

- **Greedy Depth-First Traversal:** DFS dives down a single branch of the search space as deep as it can go until it hits a dead end or a wall before backtracking.
- **Lack of Distance Consideration:** DFS does not care about path length; its only objective is to find *any* valid sequence of moves to the destination.
- **Result:** If a suboptimal branch happens to be evaluated first by the stack implementation, DFS will follow that long, winding route entirely, resulting in an inefficient path.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS vs. DFS: Memory Bottlenecks on a Massive 1000x1000 Grid

To understand why BFS and UCS struggle or crash on a massive 1000x1000 grid while DFS survives memory-wise (though failing in optimality), we look at their theoretical complexity formulas, where:

- **b** = branching factor (maximum number of valid moves from a cell).
- **d** = depth of the shallowest goal (shortest path to food).
- **m** = maximum depth of the state space.

1. Breadth-First Search (BFS) Memory Bottleneck

- **Space Complexity:** O(b^{d+1})
- **Why it crashes:** BFS must store the **entire frontier** (all unexpanded nodes at the current and next depth levels) in memory simultaneously, alongside the reached set.
- **Impact on a 1000x1000 grid:** If the food pellet is located far away, the number of nodes grows exponentially. Even with a small branching factor of b = 3 or 4, the frontier quickly consumes gigabytes of RAM, resulting in an **Out Of Memory (OOM) crash** long before the search finishes.

2. Depth-First Search (DFS) Memory Bottleneck

- **Space Complexity:** O(b*m)
- **Why it avoids crashing:** DFS only needs to store the nodes along the **single current path** from the root to the leaf node currently being explored, plus the remaining unexpanded siblings at each level of that path.
- **Impact on a 1000x1000 grid:** Because memory usage scales linearly with maximum depth (m) rather than exponentially, DFS has a remarkably small

memory footprint. It will rarely crash due to memory exhaustion on a 1000x1000 grid.

- **The Catch:** While DFS passes the memory test, its **time complexity** is O(b^m). On a 1000x1000 grid, DFS can wander down a massive, deep dead-end branch for an astronomical amount of time before finding the target.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.

FACULTY OF COMPUTING